6.5. MODULE LEVEL CONCEPTS

In previous section, we have discussed the concepts of designing the software system. We have discussed that the system can be divided into various components usually known as modules. We also defined the module as the logically separable part of the system which are discrete and independent. Here, the independent module means that it can be implemented separately considering the effect of its implementation on other modules. Also, while dividing the system in modules, care is taken that each module supports the well defined abstraction and can be solved and modified separately. In modular design, there are two concepts which are very important to consider for an effective design of the software system.

1. Coupling
2. Cohesion.

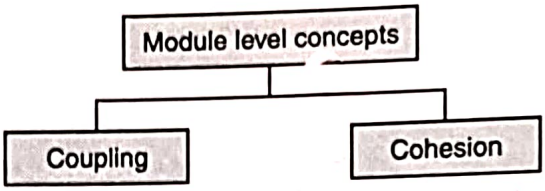

Fig. 6.7. Module level concepts.

1. Coupling : If the modules partitioned from a system are independent of each other then these modules can be implemented and modified separately without their effect on one another. But, as mentioned earlier, various modules partitioned from a system are not independent of each other. There are interconnections between the modules and these interconnections between the modules become the basis of term coupling. *So, formally, coupling between two modules is the degree of interdependence between them.* The interdependence between the modules may be in the form of parameters passed, global data, external data access, access to other modules to change data or control etc. These types of coupling are explained in next section. The modules of a system can be loosely coupled or highly coupled. The highly coupled modules have strong interconnections between them whereas the loosely coupled modules have weak interconnections between them.

The below diagram shows the coupling between the four modules. In the first diagram, all the four modules are independent of each other and there is no coupling between them. In the second diagram, the modules are loosely coupled. In third diagram, the modules are strongly coupled.

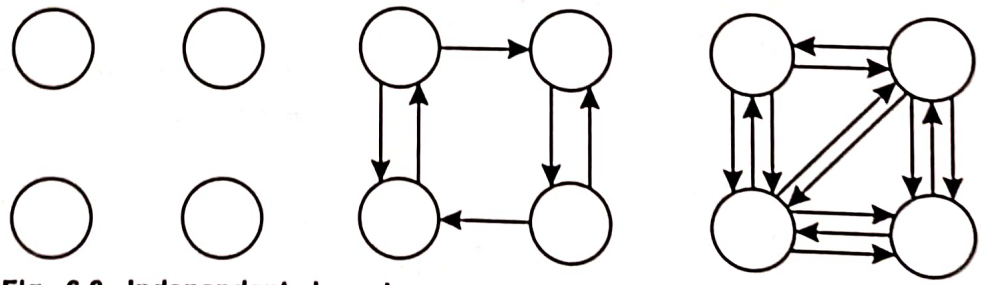

Fig. 6.8. Independent, loosely coupled and strongly coupled modules.

The software system design is said to be the best design if the modules are independent but as already mentioned, completely independent modules are impossible to have in a system. So,

a software system design is said to be the good design if the modules are loosely coupled. Coupling can be categorized in different categories as :

(*i*) Content coupling
(*ii*) Common coupling
(*iii*) External coupling
(*iv*) Control coupling
(*v*) Stamp coupling
(*vi*) Data coupling.

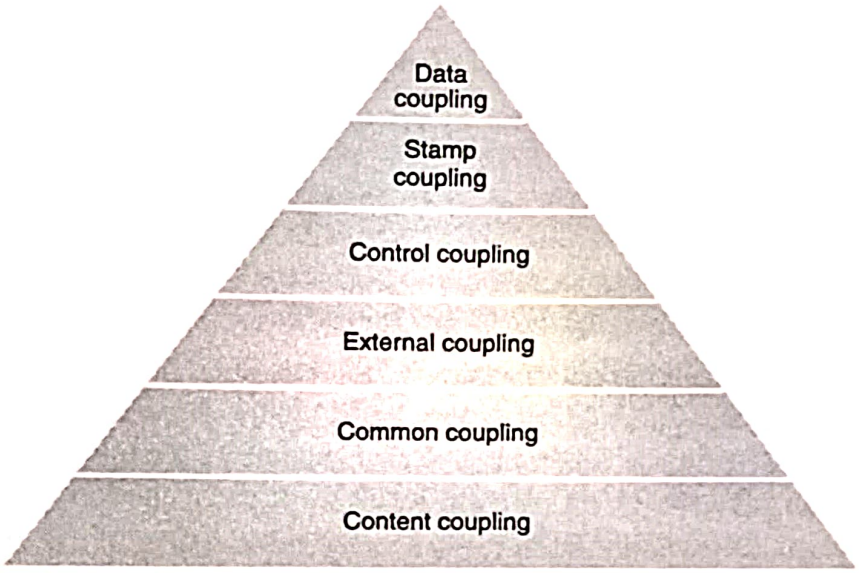

Fig. 6.9. Types of coupling.

(*i*) Content coupling : When one module can directly access the content of other modules then it is said to be the content coupling. In this type of coupling, one module can modify the data of another module and is considered as the worst type of coupling.

(*ii*) Common coupling : As the name indicates, the modules have access to some common data. This means there is some data (global data) which is shared between the various modules of the system. This coupling is also termed as *global coupling*.

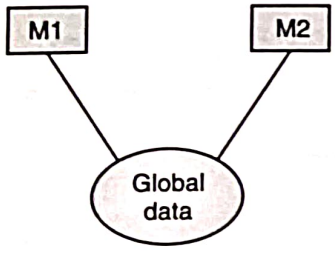

Fig. 6.10. Common coupling.

(*iii*) External coupling : In external coupling, the module depends upon the modules which are external to the system being developed. The module communicates to the external tools or external file or devices which are not the part of the system being developed.

(iv) Control coupling : When one module decides the functioning of other module then the interdependence is said to be control coupling. The module may change the flow of execution of the other module.

(v) Stamp coupling : When the modules share the common data structures and work on the different parts of it then the coupling is termed as stamp coupling. Here, a complete non-global data structure is passed from one module to another module.

(vi) Data coupling : When the modules communicate with each other by passing data as parameters then such type of coupling is said to be the data coupling. In this type of coupling, the modules are assumed to be independent and work independently. But, if they need to communicate then the communication is done by passing parameters.

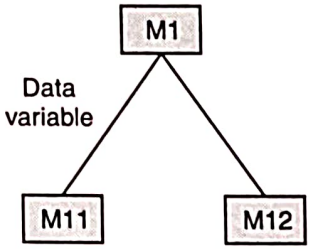

Fig. 6.11. Data coupling.

2. Cohesion : This is the degree of togetherness or degree of intra-dependability between the elements of the module. Cohesion represents how tightly the elements of the module are bound to each other to perform a single task of that module. Cohesion can be assumed to be glue that keeps all the elements of the module together. If the cohesion of all the modules is more then there will be less coupling between the modules. There is no defined relationship but it has been observed from the study of historical projects.

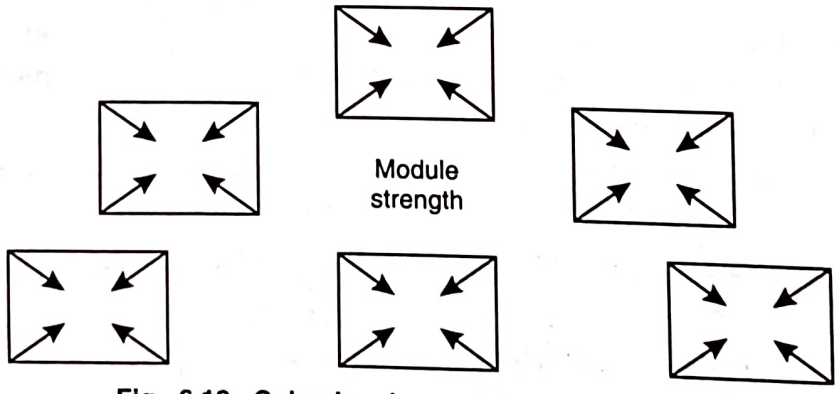

Fig. 6.12. Cohesion in modules of a system.

There are seven types of cohesions :

(i) Coincidental cohesion

(ii) Logical cohesion

(iii) Temporal cohesion

(iv) Procedural cohesion

(v) Communicational cohesion

(vi) Sequential cohesion

(vii) Functional cohesion.

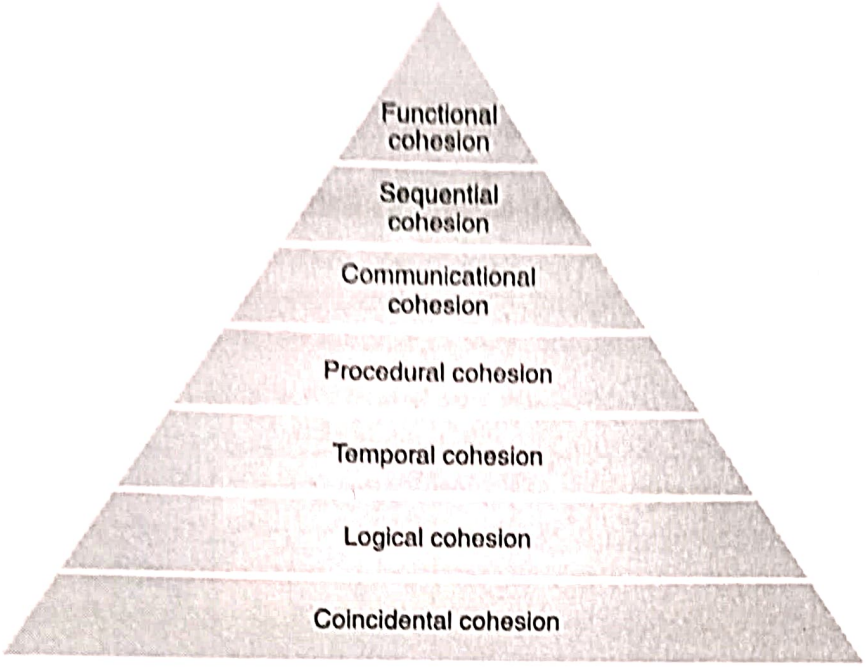

Fig. 6.13. Seven types of cohesion.

(*i*) Coincidental cohesion : Elements of the module are not related to each other. If some relation between elements occurs then it is not meaningful at all. It is worst type of cohesion in the module. Coincidental cohesion occurs when modules of the software system are created without any planning and modules are created just by chopping into pieces.

(*ii*) Logical cohesion : When there is some logical relationship between the elements of the module then it is called logical cohesion. The functions performed by the elements of the module come under same logical class. For example, a module performs all the input, output and printing functions come under logical cohesion as it performs the operations of same category *i.e.* input or output functions. If we want to input or output or print some data then we need to call this module by conveying it using some parameters.

(*iii*) Temporal cohesion : Temporal cohesion is specialized form of logical cohesion as it also contains logically similar functions. The difference is that it contains the functions which are related in time and are executed together. The functions like initialization, termination etc. are temporally bound and come under temporal cohesion.

(*iv*) Procedural cohesion : If the module contains the elements that belong to a common procedural unit then the cohesion available in that module is termed as procedural cohesion. The elements of such a module execute sequentially in order to perform a task. The module contains a complete function or several functions. For example, module containing the function to calculate marks of the students.

(*v*) Communicational cohesion : When a module contains the elements which work on the same data then the module is said to have communicational cohesion. In communicational

cohesion, the elements operate on the same input and output data. For example, a module that sends the record to printer has communicational cohesion.

(vi) Sequential cohesion : When the elements of the module are grouped in such a way that the output of one element becomes the input of other element in the same module then the module is said to have sequential cohesion.

(vii) Functional cohesion : When all the elements of the module are grouped to perform a single function then the module is said to have the functional cohesion. This is the strongest cohesion and such modules can be reused without or with a little hurdle.

Cohesion and coupling are related to each other. It has been observed that if the cohesion within each module is more, the coupling will be less between the modules. For a better design the coupling between the modules of the system should be minimum and cohesion within each module should be maximum.

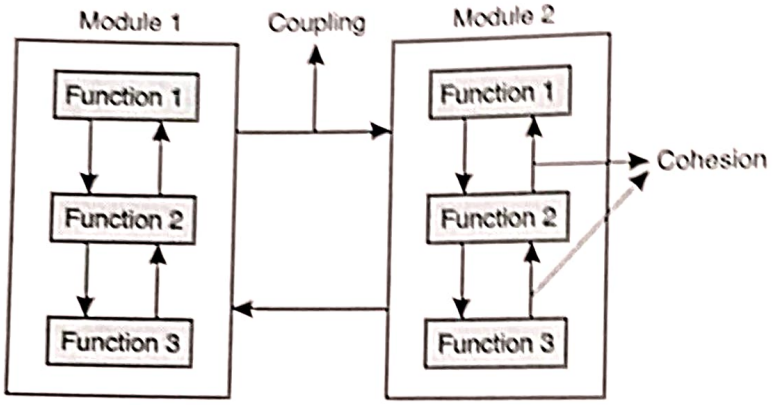

Fig. 6.14. Cohesion and coupling.

Difference between coupling and cohesion

Coupling	Cohesion
It is the degree of interdependence between the modules of the software system.	It is the degree of togetherness or degree of intra-dependability between the elements of the module.
It is also referred as inter-module binding.	It is also referred as intra-module binding.
It is the relationship between the various modules of the system.	It is the relationship between the elements of the module.
For a good design, the coupling between the modules must be minimum. That is, the dependence between the modules must be minimum.	For a good design, the cohesion within the module must be maximum. That is, the elements of the module must be grouped to perform a single task.

6.6. DATABASE DESIGN CONCEPTS